Laboratory Manual

Computer Programming Laboratory

CSE 162 (V1, V2)

EEE/ECE 103IL (V3)

Department of Electrical & Electronic Engineering (EEE)
School of Engineering (SoE)
Brac University

Revision: May 2024

A. Course Objectives:

The objectives of this course are to:

- introduce algorithms and flowchart construction
- teach students the basic syntax of a programming language (variables, data types, operators, expressions, assignments, etc.)
- explain how to solve basic programming related problems
- determine syntax and semantic errors in a program
- introduce Integrated Development Environments(IDE)s as tools for solving programming problems

B. Course Outcomes, CO-PO-Taxonomy Domain & Level- Delivery-Assessment Tool:

Sl.	CO Description	POs	Bloom's taxonomy domain/ level	Delivery methods and activities	Assessment tools
EEE 103 Computer Programming					
CO1	**Write** algorithms, flowcharts to solve basic and complex programming problems	a	Cognitive/ Apply	Lectures, notes	Quiz, Exam
CO2	**Implement** conditional statements, and loops and perform tracing to solve programming tasks	a	Cognitive/ Apply	Lectures, notes	Quiz, Exam, Assignment
CO3	**Apply** the concepts of array, string, functions, pointer, and memory addressing in programming	a	Cognitive/ Apply	Lectures, notes	Quiz, Exam, Assignment
EEE 103IL Computer Programming Laboratory					
CO4	**Use** IDE tools to compile and execute programs	e	Cognitive/ Apply, Psychomotor/ Manipulation	Lab class	Lab Work, Lab Exam, Project

C. Mark Distribution (Tentative, Subject to Change)

Assessment Tools	Weightage
Class Performance and Attendance	10%
Lab Report	20%
Project	25%
Software Test	25%
Final Lab Quiz	20%

CO Assessment Plan:

Assessment Tools	Course Outcomes
	CO4
Project	x
Lab Test	x

D. References

Sl.	Title	Author(s)	Publication	Edition	Publisher	ISBN
1	Teach Yourself C	Herbert Schildt	1997	3rd	McGraw-Hill Osborne Media	978-0078823114
2	Let Us C	Yashavant Kanetkar	2016	15th	BPB Publications	978-8183331630

Lab Safety and Security Issues

1. *Laboratory Safety Rules (General Guidelines):*

The Department of EEE maintains general safety rules for laboratories. The guideline is attached in front of the door in each of the laboratories. The written rules are as follows.

1. Always check if the power switch is off before plugging into the outlet. Also, turn the instrument or equipment OFF before unplugging from the outlet.
2. In case of fire, disconnect the electrical mains power source if possible.
3. Students must be familiar with the locations and operations of safety and emergency equipment like Emergency power off, Fire alarm switches, and so on.
4. Eating, drinking, chewing gum inside electrical laboratories are strictly prohibited.
5. Do not use damaged cords or cords that become too hot or cords with exposed wiring and if something like that is found, inform the teacher/LTO right away.
6. No laboratory equipment can be removed from their fixed places without the teacher/LTO's authorization.

2. Electrical Safety:

To prevent electrical hazards, there are symbols in front of the Electrical Distribution Board, High voltage three-phase lines in the lab, Backup generator, and substation. Symbols related to Arc Flash and Shock

Hazard, Danger: High Voltage, Authorized personnel Only, no smoking, etc. are posted in required places. Only authorized personnel are allowed to open the distribution boxes.

3. Electrical Fire:

If an electrical fire occurs, try to disconnect the electrical power source, if possible. If the fire is small, you are not in immediate danger, use any type of fire extinguisher except water to extinguish the fire. When in doubt, push the Emergency Power Off button.

4. IMPORTANT:

Do not use water on an electrical fire.

List of Experiments

Exp. No.	Experiment Name	Page	Tentative schedule	Is this experiment used for any CO assessment?	
				Yes	No
1	Introduction to Algorithm and Flowcharts	1-8	2nd week	✔	
2	Introduction to C programming, I/O, Datatypes	9-16	3rd week	✔	
3	Familiarization with Decision Control Structures & Loop Control Structures	17-25	4th week	✔	
4	Familiarization with Functions and Recursion	26-32	5th week	✔	
5	Arrays and Strings in C Language	33-38	6th week	✔	
6	Pointers and Dynamic Memory Allocation	39-43	7th week	✔	
7	Familiarization with Structure	44-48	8th week	✔	
	Project		9th-12th week	✔	

Updated by:

1. Raihana Shams Islam Antara , Tasfin Mahmud (Experiment 1)
2. Md Rakibul Hasan (Experiment 5)
3. Md. Nahid Haque Shazon (Experiment 4)
4. Md. Mahmudul Islam (Experiment 3)
5. Taiyeb Hasan Sakib (Experiment 6)
6. Shahed-E-Zumrat (Experiment 1 and 2)

Rubrics (CO4) for Lab Test

Performance Criteria	Marks	Performance Measure			
		Unsatisfactory (20%)	**Developing (50%)**	**Satisfactory (80%)**	**Exemplary (90%)**
Familiarization with the structure and syntax of IDE	7	Not familiar with the structure and syntax of IDE appropriately	Familiar with the structure and syntax of IDE with limitations	Familiar with the structure and syntax of IDE	Proficient with the syntax and structure of IDE
Use IDE to execute program effectively	7	Inadequacy in executing program for desired purpose	Cannot obtain all desired outputs but able to complete part of tasks	Can obtain desired outputs but less efficiency in programs	Can obtain desired outputs with efficient programs
Development of efficient algorithm for IDE to obtain desired solution	4	Unable to develop algorithm or integrate the algorithm with IDE	Able to develop algorithm but limitation in integration with IDE	Able to develop algorithm for IDE to obtain desired solution	Shows proficiency in the development of efficient algorithm for IDE to obtain desired solution
Use IDE to implement	7	Unable to obtain any	Obtain solution using IDE with	Obtain solution using IDE with	Obtain solution using IDE

algorithm and execute solution		solution using IDE	significant errors	limited number of errors	without any error

Experiment No. 1

Introduction to Algorithm and Flowcharts

Objective:
The objective of this lab is to understand the concepts of algorithms and flowcharts and how they are used to design and represent a program's logic before coding in the C programming language.

Learning Outcome:
Upon completion of this experiment, the students shall be able to:

1. Define algorithms and create step-by-step procedures for problem-solving.
2. Use flowchart symbols to visually represent algorithms and logical sequences.
3. Analyze problems, develop logical solutions, and effectively test/debug C programs.

Tools Required:

- Pen and Paper (for manual drawing of flowcharts)
- Software for creating flowcharts (optional, e.g., Lucidchart, draw.io)

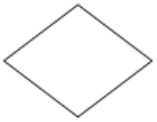
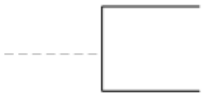
Introduction:
Algorithms and flowcharts serve as foundational tools for effective problem-solving and program development. An algorithm is a step-by-step procedure or formula for solving a problem, offering programmers a clear and concise method to follow. It breaks down complex problems into manageable tasks, ensuring each step is logically sound and sequentially correct. A flowchart, on the other hand, is a visual representation of an algorithm. It uses standardized symbols to depict the flow of control and data through the program, making it easier to understand and communicate the structure of a program. Together, algorithms and flowcharts help programmers plan, visualize, and troubleshoot their code, fostering a deeper understanding of programming logic and improving the overall efficiency and reliability of the software development process.

Properties of Algorithm:

1. **Finiteness:** An algorithm must always terminate after a finite number of steps. This means that after every step, one reaches closer to the problem's solution, and after a finite number of steps, the algorithm reaches an end point.
2. **Definiteness**: Each step of an algorithm must be precisely defined. This is done by well-thought-out actions that must be performed at each step of the algorithm. Also, the actions are defined unambiguously for each activity in the algorithm.
3. **Input:** Any operation you perform needs some beginning value/quantities associated with different operations activities. So the value/quantities are given to the algorithm before it begins.
4. **Output:** One always expects output/result (expected value/quantities) in terms of output from an algorithm. The result may be obtained at different stages of the algorithm. If some result is from the intermediate stage of the operation, then it is known as intermediate result, and result obtained at the end of the algorithm is known as end result. The output is the expected value/quantities that always have a specified relation to the inputs.

5. **Effectiveness:** Algorithms to be developed/written using basic operations. Actually, operations should be basic, so that even they can in principle be done exactly and in a finite amount of time by a person, using paper and pencil only.

Shapes and Symbols Used in Flowcharts:

Flowchart Symbol	Symbol Name	Description
	Terminal (Start or Stop)	Terminals (Oval shapes) are used to represent start and stop of the flowchart.
	Flow Lines or Arrow	Flow lines are used to connect symbols used in flowchart and indicate direction of flow.
	Input / Output	Parallelograms are used to read input data and output or display information
	Process	Rectangles are generally used to represent process. For example, Arithmetic operations, Data movement etc.
	Decision	Diamond shapes are generally used to check any condition or take decision for which there are two answers, they are, yes (true) or no (false).
	Connector	It is used connect or join flow lines.
	Annotation	It is used to provide additional information about another flowchart symbol in the form of comments or remarks.

Steps to Create an Algorithm and Flowchart:

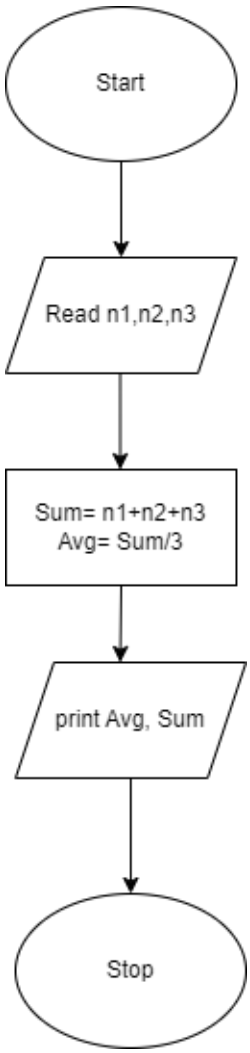
1. **Define the problem**: Understand the problem statement clearly.
2. **Develop an algorithm**: Write down the step-by-step instructions to solve the problem.
3. **Draw the flowchart**: Represent the algorithm using flowchart symbols and connect them with arrows.

Example Problem 1: Write an algorithm and flowchart to find the sum and average of three numbers.

Flowchart:

Algorithm:

Step 1: Start
Step 2: Declare variables num1, num2, num3, sum, and average.
Step 3: Read values num1, num2, and num3.
Step 4: Add num1, num2, num3, and assign the result to sum.
sum←num1+num2 +num3
average ← sum/3
Step 5: Display the sum and average
Step 6: Stop



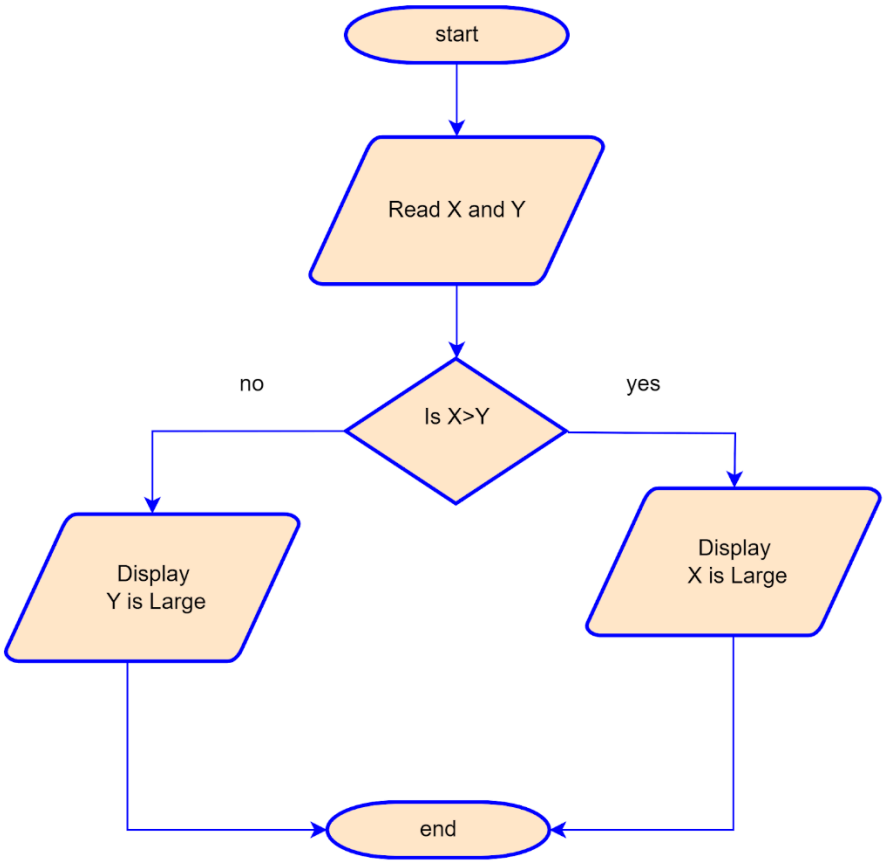
The ***branch*** refers to a binary decision based on some condition. If the condition is true, one of the two branches is explored; if the condition is false, the other alternative is taken. This is usually represented by the 'if-then' construct in pseudo-codes and programs. In flowcharts, this is represented by the diamond-shaped decision box. This structure is also known as the selection structure.

Example Problem 2: Write an algorithm and Flowchart to Find the Largest of Two Numbers

Algorithm:

Step 1: Start
Step 2: Read X, Y
Step 3: Is X>Y
Print X is large
Else Print Y is large.
Step 4: Stop

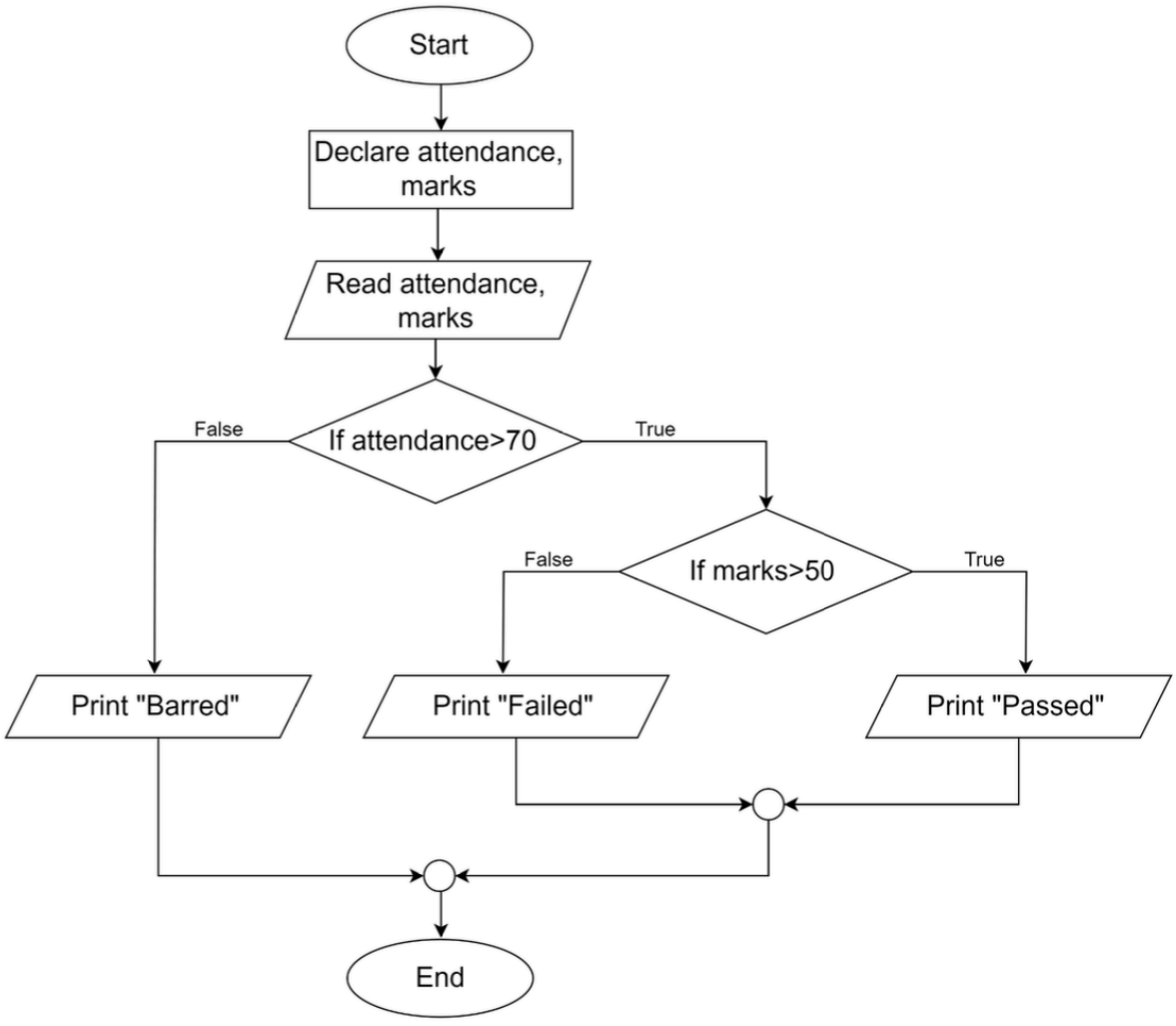
Flowchart:



Example Problem 3: Suppose you are tasked to submit grades of EEE-103. You will ask for a total of 2 numbers: the attendance percentage and the total obtained marks. If the attendance percentage is greater than 70% and the student has obtained more than 50 marks then the program will print "Passed" otherwise "Failed". If attendance is less than 70% no need to check the marks and print "Barred".

Draw a flowchart for the above problem.

Flowchart:



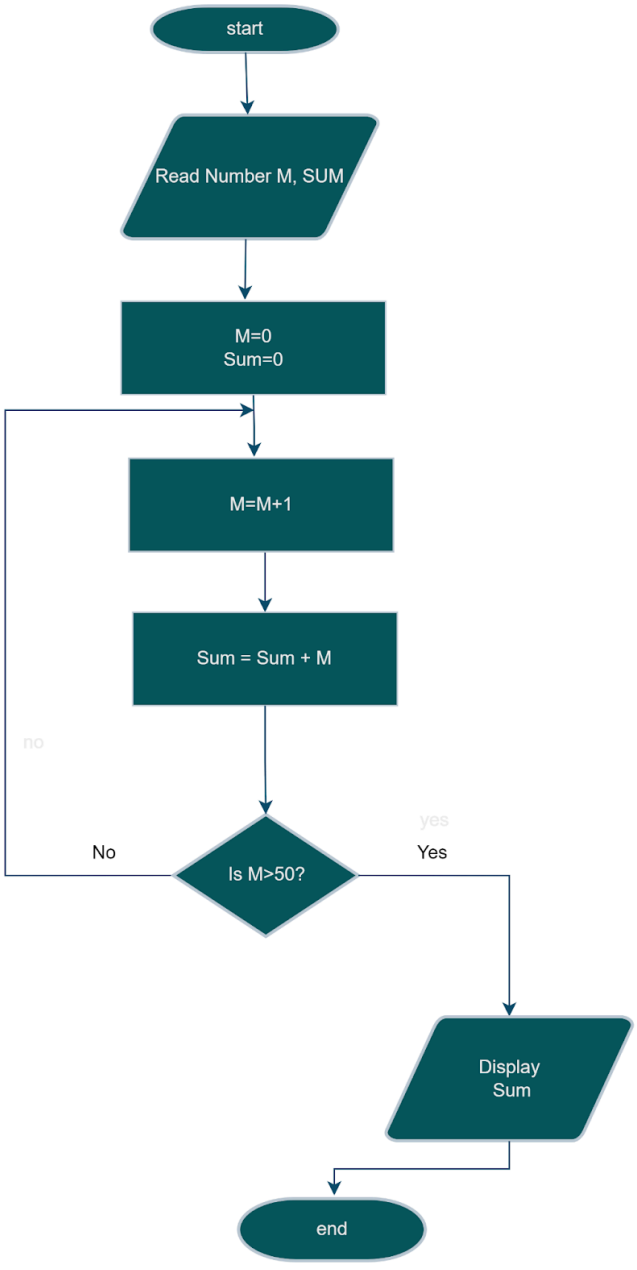
Loop: The *loop* allows a statement or a sequence of statements to be repeatedly executed based on some loop condition. It is represented by the 'while' and 'for' constructs in most programming languages. In the flowcharts, a back arrow hints at the presence of a loop. A trip around the loop is known as iteration. You must ensure that the condition for the termination of the looping must be satisfied after some finite number of iterations, otherwise, it ends up as an infinite loop, a common mistake made by inexperienced programmers. The loop is also known as the repetition structure.

Example Problem 4: Write an algorithm and Flowchart to calculate the Sum of The First 50 Numbers

Algorithm:

Step 1: Read number M and Sum
Step 2: Declare number M= 0 and Sum= 0
Step 3: Determine M by M= M+1
Step 4: Calculate the sum by the formula: Sum= Sum + M
Step 5: Add a loop between steps 3 and 4 until M= 50.
Step 6: Print Sum

Flowchart:

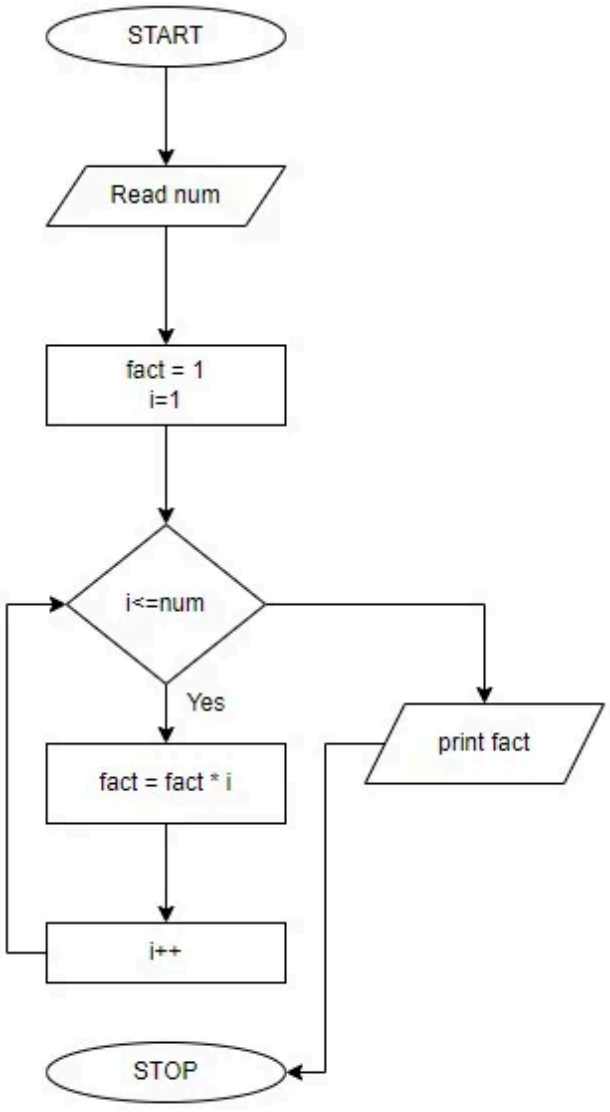


Example Problem 5: Write an algorithm and draw the flowchart to find the factorial of a number.

Algorithm:

Step 1: Start
Step 2: Read a number: num
Step 2: Initialize variables: i = 1, fact = 1
Step 3: if i <= n go to step 4, otherwise go to step 6
Step 4: Calculate: fact = fact * i
Step 5: Increment the i by 1 (i=i+1) and go to step 3
Step 6: Print fact
Step 7: Stop

Flowchart:



Lab report:

1. Draw a flowchart for a program that calculates and prints the grade for a student based on their score. First ask the user for the obtained marks. Determine the grade based on the following grading scale:

90 or above: A
80 - 89: B
70 - 79: C
60 - 69: D
Below 60: F

Print the grade.

2. Write an algorithm and draw the flowchart of a program that reads the radius of a circle and prints its circumference and area. If the area is larger than 100 print “Too large”, if the area is smaller than 50 print “Too small”; otherwise print “Perfect”.

3. Write an algorithm and draw the flowchart of a program that reads five numbers as input from the user, prints whether the numbers are odd or even and prints the average of the numbers.

4. Write an algorithm and flowchart of the following program: You are tasked with writing a program that calculates the total bill for a customer's purchase. The program should perform the following steps: Prompt the user to enter the prices of three items (item1, item2, and item3). Calculate the subtotal and add a 10% VAT to the total. If the total bill (including VAT) is more than 500, apply a 50 TK discount.

Print the total bill after applying the VAT and discount.

Experiment No. 2

Introduction to C programming, I/O, and Datatypes

1. **Objective:** The Lesson is designed to teach the fundamental programming concepts and methodologies in the C programming language via laboratory experiences. CodeBlocks is the Integrated Development Environment (IDE) that will be used.

2. **Learning Outcome:** Upon completion of this experiment, the students shall be able to:
   • Write C programs on CodeBlocks
   • Perform input/output(I/O) operations using C programming language.
   • Define data types and use them in simple data processing applications

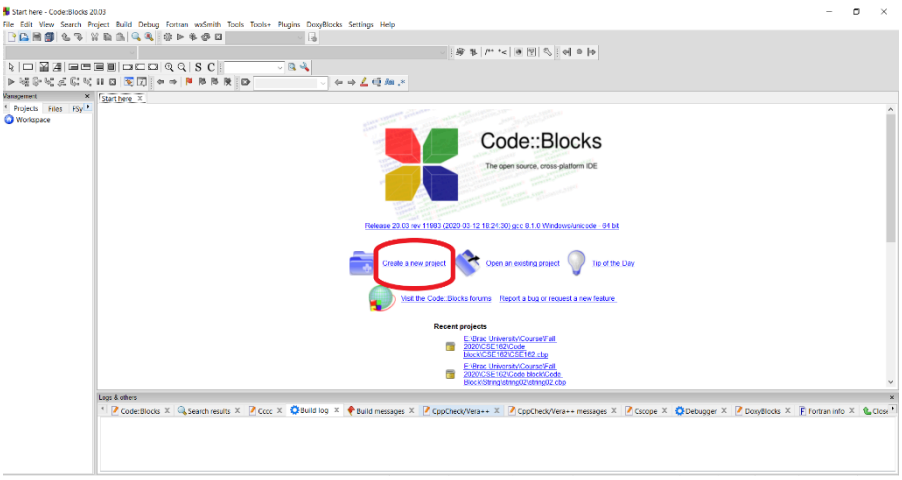
3. **Software requirements:**
   - Any IDE that supports a C compiler (preferably Code::Blocks; any other IDE or text editor with a C compiler installed will also be accepted.)

4. **Example Problems**

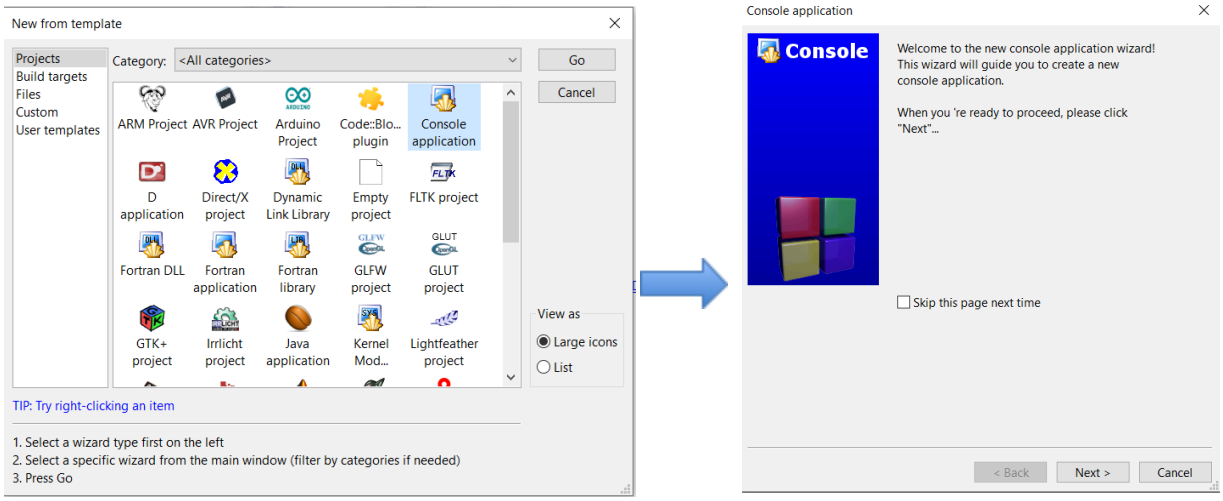
Example 1: Writing C programs in CodeBlocks

To start a new project click on

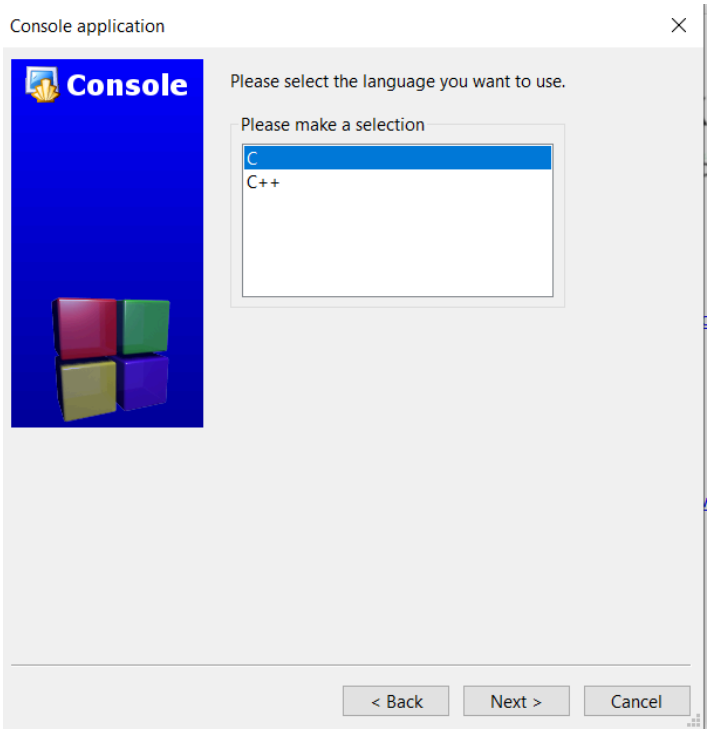
File >New >Project Or click “Create a new project”



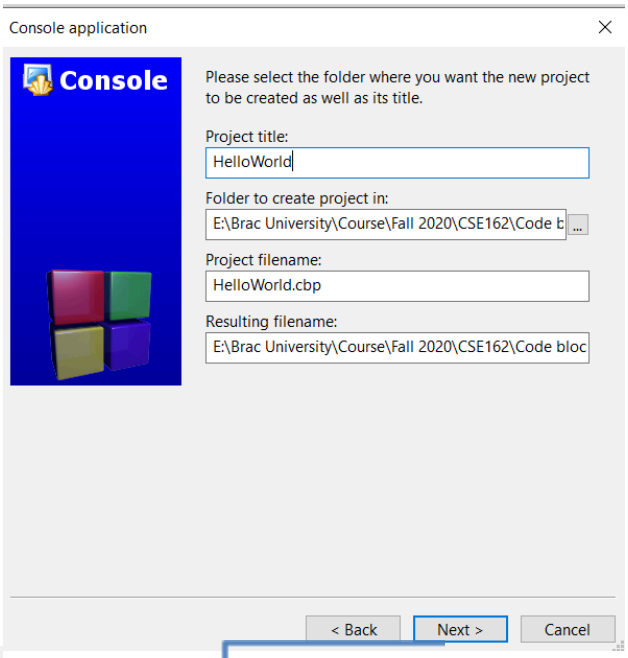
Select Console application, as this is the most common for general purposes, and click Go.

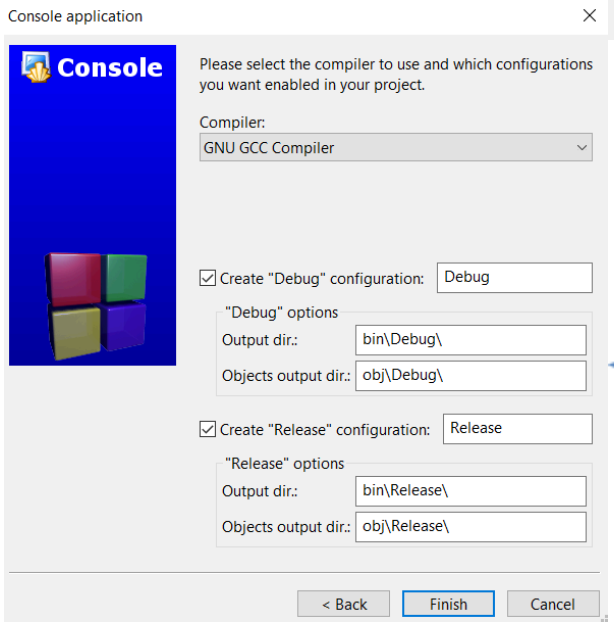


Continue through the menus, select C when prompted for a language



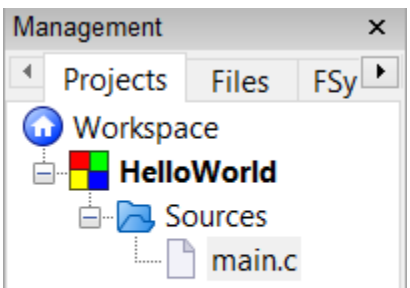
In the next screen, give the project a name and select a destination folder. CodeBlocks will generate the remaining entries from these two.





Press finish and the project will be generated.

The main window will turn gray, the source file needs only to be opened. In the Projects tab of the Management pane



on the left expand the folders and double click on the source file main.c to open it in the editor.

Code:

```
#include <stdio.h>
#include <stdlib.h>

//How to add comments- This is a way
/*This is my first lab, so I create
a test file – this is another way*/

int main()
{
	printf("Hello world!\n");

	//this is "\n" for creating a new line

	//this is "\t" for creating a tab

	//To print the value of a variable to the screen,

	//we will use the printf() function.

	//we will at first print I like Programming

	printf ("I like programming");

	return 0;
}
```

Example 3: Use of Conversion Specifiers

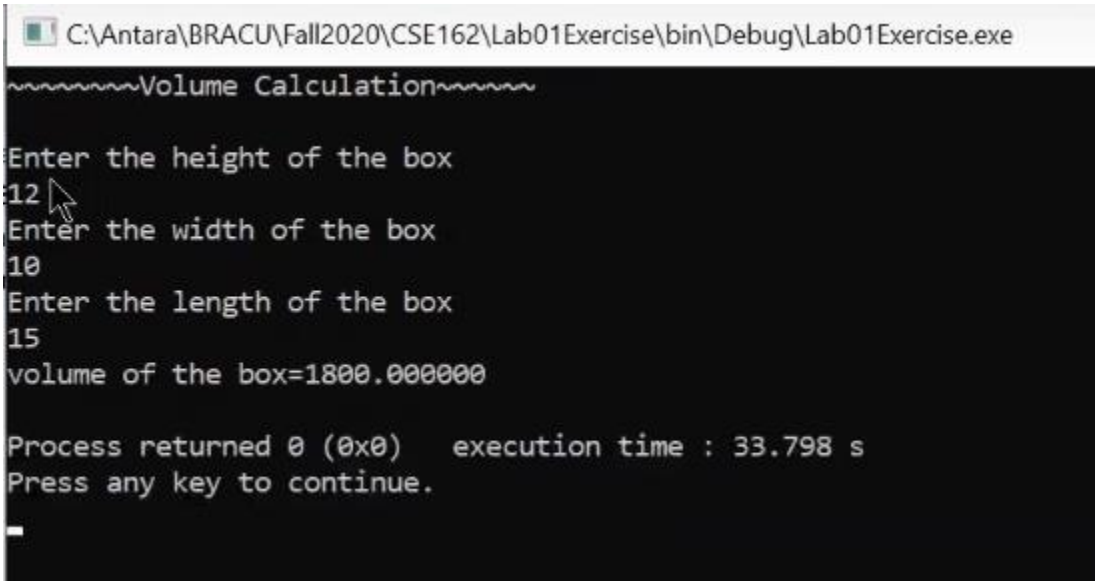
Code:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int try_integer = -67;
    unsigned int try_ui = 33;
    float try_float = 9.82;
    double try_double = 573717.1817818;
    char try_char = 'y';
    char try_string[] = "Let's try to print it";

    printf ("Integer: %d\n", try_integer);
    printf ("Unsigned integer: %u\n", try_ui);

    printf ("Floating-point: %f\n", try_float);
    printf ("Double, exponential notation: %17.11e\n", try_double);

    printf ("Single character: %c\n", try_char);
    printf ("String: %s\n", try_string);

    printf("%s received the %s award this year\n", "John", "Leadership" );
    printf("I took %d courses in this semester\n", 4);
    printf("The pi value is %f\n", 3.1415924653);
    printf("The pi value is %.2f", 3.1415924653);

    return 0;
}
```

Example 4: Write a program in C to calculate and display the volume of a box based on the user inputs length, width and height.

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("~~~~~~~~Volume Calculation~~~~~~\n\n");
    int h;
    int w, l;
    float vol;
    printf("Enter the height of the box\n"); scanf("%d", &h);
    printf("Enter the width of the box\n"); scanf("%d", &w);
    printf("Enter the length of the box\n"); scanf("%d", &l);
    vol=h*w*l;
    printf("volume of the box=%f\n", vol);
    return 0;
}
```

Output:



5. **Results after running the example programs:**
   Screenshots of the results:

6. **Lab report assignments (submit source code and screenshots of results):**

Task 1: First find the value of each of the following arithmetic expressions in C. Then step by step, show in detail, how to hand calculate those values? Simply, you will write your investigation report on how C in CodeBlocks found those answers.

For example, if you are told to write 5+4*3/2-1, Your answer should be like:

= 5+4*3/2-1 **=** 5+12/2-1 **=** 5+6-1 **=** 11-1 **=** 10, in the investigation report

A) 3+4.8*2
B) 12%16
C) 2.*6/5
D) 5.2+12/8
E) 4 -(6+18.0/3)/3
F) 17-12*4
G) 2*6/5.
H) 7/(4*2)
I) 2*6/5
J) 19%13
K) 36/60

Task 2: Try the following programs and note what happens. Then try to explain the behaviors line by line.

```
int main()
{
   char  b;
int  i = 257;
double d = 323.142;
b = i;
printf("%c\n",b);
b = (char) i;
printf("%c\n",b);
i = (int) d;
printf("%d\n",i);
b = (char) d;
printf("%c\n",b);

}
```

Task 3: Here is a pseudo program that computes the number of miles that light will travel in a specified number of days. Try to compile and run the following C program.

If it does not work, make necessary corrections so that it compiles, runs and gives correct output.

```
Lightspeed = 186000
days = 1000;            //specify number of days here
seconds = Days*24*60*60;    /*convert to seconds/
distance = lightspeed * Second;       //compute distance
printf("In " + days + " days light will travel about ");
printf(Distance + "miles.");
```

Task 4) Write a program in C that can calculate the area of a circle up to 4 decimal points taking the radius as input. Take the value of radius from the user in the console window and radius can be any integer or decimal fraction number. Print the value of the area.
(Submit your code in report by copy pasting it and also attach a screenshot of your result in command window)

7. **Comment/Discussion on the obtained results and discrepancies (if any).**

Experiment No. 3

Familiarization with Decision Control Structures & Loop Control Structures.

1. **Objective:** This experiment is designed to demonstrate the use of control statements and loop structures and its effectiveness in C programming

2. **Software requirements:**
    - Any IDE that supports a C compiler (preferably Code::Blocks; any other IDE or text editor with a C compiler installed will also be accepted.)

3. **Theoretical Background and Example Problems**

    Decision Control Structure:
    While writing programs, we often want a set of instructions to be executed in one situation, and an entirely different set of instructions to be executed in another situation.
    This kind of situation is dealt with in C programs using a decision control instruction. A decision control instruction can be implemented in C using:
    (a) The if statement
    (b) The if-else statement
    (c) The conditional operators

    Example 1: Write a program to calculate the salary as per the following table:

Gender	Years of Service	Qualification	Salary
Male	>= 10	Post - Graduate	15000
	>= 10	Graduate	10000
	<10	Post - Graduate	10000
	<10	Graduate	7000
Female	>= 10	Post - Graduate	12000
	>= 10	Graduate	9000
	<10	Post - Graduate	10000
	<10	Graduate	6000

    Ideation: First of all, we have 2 different salary structures based on the gender. So an ***if else if*** block will be used to program salary structures for male and female employees. Then we have to insert one ***if-else*** block in each of the blocks for male and female employees which will be used for employees

with different ranges of 'years of service'. Finally another ***if-else*** block will be inserted in each of the previous if and else blocks to deal with different level of qualifications. Let us take a look at the code.

Code:

```
#include<stdio.h>
#include<stdlib.h>


int main()

{  char gen;
   int yos, sal, qual;
   printf("Enter the gender of the employee. M for male and F for
female:
   ");

   scanf(" %c",&gen);
   printf("\nEnter the years of service of the employee: ");
   scanf("%d", &yos);
   printf("\nEnter the qualification of the employee. 0 for graduate
   and 1
   for post-graduate: ");

   scanf("%d", &qual);
   printf("\n\n");


   if (gen == 'M')

   {
      if (yos >= 10)
      {
           if (qual == 1)
           {
                 sal = 15000;
           }
           else if (qual == 0)
           {
                 sal = 10000;
           }
      }

     else
     {
           if (qual == 1)
           {
                sal = 10000;
```

```
            }
            else if (qual == 0)
            {
                 sal = 7000;
            }
      }
   }


   else if (gen == 'F')
   {
      if (yos >= 10)
      {
            if (qual == 1)
            {
                 sal = 12000;
            }
            else if (qual == 0)
            {
                 sal = 9000;
            }
      }
      else
      {
            if (qual == 1)
            {
                sal = 10000;
            }
            else if (qual == 0)
            {
               sal = 6000;
            }
      }
   }
   printf("Salary is %d\n\n", sal);
   return 0;


}
```

Example 2: If the three sides of a triangle are entered through the keyboard, write a program to check (a) Whether the triangle is valid or not. (The triangle is valid if the sum of two sides is greater than the largest of the three sides.)
(b) Check whether the triangle is isosceles, equilateral, scalene or right angled triangle.

Ideation: We have to scan the length of 3 sides from the user and at first check for the validity of the triangle. An ***if-else*** block will be used for this purpose. If the lengths given form a valid triangle, we can check for whether at least two of the sides are equal using another ***if-else*** block. If at least two sides are equal we can check for isosceles or equilateral using another ***if-else*** block. Otherwise it is scalene triangle. For checking whether the triangle is right-angled, we have to check whether the sides agree with Pythagorean Theorem in any of the three possible combinations using an ***if*** block.

Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
int main()
{
       int a, b, c; //Three sides of the triangle
       printf("Enter the length of the 1st side: ");
       scanf(" %d", &a);
       printf("\nEnter the length of the 2nd side: ");
       scanf(" %d", &b);
       printf("\nEnter the length of the 3rd side: ");
       scanf(" %d", &c);


       printf("\n\n");


       if ((a+b > c) && (b+c > a) && (c+a > b))
       {
           if ((a == b) || (b == c) || (c == a))
           {
                if ((a == b) && (b == c))
                {
                     printf("Equilateral Triangle\n");
                }
               else
                {
                     printf("Isosceles Triangle\n");
                }
           }
          else
           {
                printf("Scalene Triangle\n");
           }
           int a_sq = pow(a, 2);
           int b_sq = pow(b, 2);
           int c_sq = pow(c, 2);
```

```
        if ((a_sq == b_sq + c_sq) || (b_sq == c_sq + a_sq) ||
(c_sq == a_sq +b_sq))
        {
             printf("Right-angled Triangle\n");
        }
     }
     else
    {
           printf("Not Valid\n");
    }


}
```

Loop Control Structure:

The versatility of the computer lies in its ability to perform a set of instructions repeatedly. This involves repeating some portion of the program either a specified number of times or until a particular condition is being satisfied. This repetitive operation is done through a loop control instruction. There are three methods by way of which we can repeat a part of a program. They are:

(a) Using a for statement

(b) Using a while statement

(c) Using a do-while statement

Example 3: Fibonacci series: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55… Starting from the third term, any term in Fibonacci series is the summation of previous two terms. Write a program using while loop to display first n terms of Fibonacci series and a sum of these n terms. n is a number anywhere between 3 to 45 to be given as input by the user. You have to use ***While*** loop to solve the problem. **Ideation:** We have to set the values of two variables indicating two previous terms to 0 & 1. We have to constantly update these variables as we move forward to higher terms.

Each term will be the sum of previous two terms (i.e. the values stored in the above mentioned variables). Another variable for sum of the terms will have to be updated after each term by adding the new term to the previously calculated sum.

Code:

```
#include <stdio.h>
int main()
{
    int t1 = 0, t2 = 1;
    int sum = 1;
    int n;
    /*Ask the user to enter any number between 3 and 45
    It the number is out of this range, display an error message */
```

```
    printf("Enter the number of terms to be displayed: ");
    scanf("%d", &n);


    if (n<3 || n>45)
    {
         printf("\n\nInvalid!!\n\n");
    }
    else
    {
         printf("\nThe Fibonacci series upto %dth term:\n", n);
         printf("%d, %d", 0, 1);
         int i = 3;
         int t;
         while (i <= n)
         {
                 t = t1+t2;
                 t1 = t2;
                 t2 = t;
                 sum = sum + t;
                 printf(", %d", t);
                 i++;
         }
         printf("\n\nThe sum of the first %d terms of Fibonacci series
=
%d\n\n",n, sum);
    }
}
```

Example 4: Write a program to produce the following output:

```
A   B   C   D   E   F   G   F   E   D   C   B   A
A   B   C   D   E   F       F   E   D   C   B   A
A   B   C   D   E               E   D   C   B   A
A   B   C   D                       D   C   B   A
A   B   C                           C   B   A
A   B                               B   A
A                                       A
```

Ideation: There are many ways to generate this output. Students are encouraged to explore different ways of generating patterns. Try making a diamond using * symbols for example.

Code:

```
#include<stdio.h>
int main()
{
     for (int i = 1; i<=7; i++)
     {
        if (i == 1)
        {
               char ch = 'A';
               int k;
               for (int j = -6; j<=6; j++)
               {
                  if (j<0)
                  {
                      k = -j;
                  }
                  else
                  {
                      k = j;
                  }
                  printf("%c ", ch + (6-k));
               }
         }
       else
       {
                int num_space = 2*i - 3;
                int x = num_space/2;
                int k;
                char ch = 'A';
                for (int j = -6; j<=6; j++)
                {
                   if (j<-x || j>x)
                   {
                      if (j<0)
                      {
                         k = -j;
                      }
                      else
                      {
                          k = j;
                      }
```

```
                    printf("%c ", ch + (6-k));
                }
                else
                {
                    printf(" ");
                }
            }
        }
        printf("\n\n");
    }
}
```

4. **Results after running the example programs:**
   Screenshots of the results:

5. **Lab report assignments (submit source code and screenshots of results):**

   a. Any year is entered through the keyboard, write a program to determine whether the year is leap or not.
   b. A certain grade of steel is graded according to the following. conditions:
      (i) Hardness must be greater than 50
      (ii) Carbon content must be less than 0.7
      (iii) Tensile strength must be greater than 5600
      The grades are as follows:
      Grade is 10 if all three conditions are met
      Grade is 9 if conditions (i) and (ii) are met
      Grade is 8 if conditions (ii) and (iii) are met
      Grade is 7 if conditions (i) and (iii) are met
      Grade is 6 if only one condition is met
      Grade is 5 if none of the conditions are met
      Write a program, which will require the user to give values of hardness, carbon content and tensile strength of the steel under consideration and output the grade of the steel.
   c. Write a program for a matchstick game being played between the computer and a user. Your program should ensure that the computer always wins. Rules for the game are as follows:

      There are 21 matchsticks.

      The computer asks the player to pick 1, 2, 3, or 4 matchsticks.

      After the person picks, the computer does its picking.

      Whoever is forced to pick up the last matchstick loses the game.

d. Write a program to produce the following output:

```
1
2 3
4 5 6
7 8 9 10
```

e. Population of a town today is 100000. The population has increased steadily at the rate of 10 % per year for last 10 years. Write a program to determine the population at the end of each year in the last decade.

6. Comment/Discussion on the obtained results and discrepancies (if any).

Experiment No. 4

Familiarization with Functions and Recursion

1. **Objective:** This experiment is designed to demonstrate the use of function and its effectiveness in C programming and also demonstrates how recursion makes function definition easier.

2. **Theoretical Background:** A function is a set of statements to perform a certain task. In C program there are two types of function
    - Library function : scanf, printf, gets, puts, getch, sqrt etc.
    - User defined function

    C program contains at least one function which is main(), and main() must be present exactly once in a program. There is no limit on the number of functions defined and there can be functions which are defined but not called.

    Defining a Function:

    While declaring a function we need to specify certain things: function's name, return type, and parameters. In C programming the general form of a function definition is as follows:

```
return_type name_of_function( parameter list )
 {
   body of the function
}
```

- **Return Type** − A function may return or may not return a value. The return_type is the type of data that the function is supposed to return. If no return type is specified then default int is assumed. When the return statement is encountered: the function returns immediately and does not execute whatever is written after the return statement. Return statement can be used without return value, return; . This type of return is used mostly by void functions (no return of value). In C programming more than one value can not be returned.

- **Function Name** − It is the name of the function. Two or more functions can not have the same name, also function and variable names can not be the same.

- **Parameters** − To perform a certain task a function may need some input from the user. For this reason, to take arguments a function must have special variables known as formal parameters and parameters are like a placeholder. While invoking a function the value that is passed to the function is known as argument. Parameters are optional so there can be functions without any parameters.

- **Function Body** − The function body contains a set of statements that are designed to perform a certain task. No statements can be written outside of a function

Note: In C programming one function can not be defined inside another function but one function can be called from another function.

Function Examples:

A function definition in C programming consists of a function header and a function body. Here are all the parts of a function −

```
int sub(int x, int y)
{
  return x-y;
}
```

This function takes two integer numbers x & y and returns x-y. In main function it can be called with values as,

```
sub(2, 6)
```

Local Variables: Local variables are variables that are declared inside a function or block and they can be used only by statements within that function. Local variables of one function can not be accessed by another function or outside. The following example shows how local variables are used. Here all the variables a, b, and c are local to main() function.

For example the following code will generate error as variable i local to main only,

```
#include <stdio.h>

void myfunc(void)

{

  printf("i is %d\n", i);

}

int main(void)
```

```
{
  int i=10;
  myfunc();
  return 0;
}
```

Global Variables: Global variables are defined outside a function, usually on top of the program and they can be accessed by any of the functions.

For example the following code will generate no error as variable i is global now and can be accessed by any function.

```
#include <stdio.h>
int i;
void myfunc(void)
{
  printf("i is %d\n", i);
}
int main(void)
{
  i=10;
  myfunc();
  return 0;
}
```

Functions Calling: Functions can be called by value also by reference. Function that is called by value will have no effect on the argument used to call. On the other hand, in case of call by reference address of an argument is copied into the parameter. By default C uses 'call by value'

Returning multiple values: Multiple values can be returned using a global variable or call by reference.

Recursion: In programming languages, if a function is defined in terms of itself, then it is called a recursive function. This type of definition is also known as circular definition. Here, the function is made to call itself. When a function calls itself recursively, each invocation gets a fresh set of all the automatic variables. Recursive code is more compact and often much easier to write. To define a recursive function it must have a recursive definition and a terminating condition. An example of recursive function will be demonstrated in this experiment.

3. Software requirements:

- Any IDE that supports a C compiler (preferably Code::Blocks; any other IDE or text editor with a C compiler installed will also be accepted.)

4. Example Problems:

a. Write a function power(a, b), to calculate the value of a raised to b

Code:

```
#include<stdio.h>
int main()
{
    int num, pow;
    int val;
    printf("Enter the number: ");
    scanf("%d", &num);
    printf("\nEnter the power: ");
    scanf("%d", &pow);

    val = power(num,pow);
    printf("\nThe result is = %d", val);

    return 0;
}

int power(int a, int b)
{
    int i, res = 1;
    for (i = 1; i <= b; i++)
    {
        res = res*a;
    }
    return res;
}
```

b. A positive integer is entered through the keyboard. Write a function to obtain the prime factors of this number. For example, prime factors of 24 are 2, 2, 2 and 3, whereas prime factors of 35 are 5 and 7.

Code:

```
#include <stdio.h>

int main()
{   int i, num, isPrime;

    /* Input a number from user */
    printf("Enter any number to print Prime factors: ");
    scanf("%d", &num);

    printf("All Prime Factors of %d are: \n", num);

    /* Find all Prime factors */
    for(i=2; i<=num; i++)
        {
            /* Check 'i' for factor of num */
            if(num%i==0)
                {
                /* Check 'i' for Prime */
                isPrime = check_prime(i);

                /* If 'i' is Prime number and factor of num */
                if(isPrime==1)
                    {
                    printf("%d\n", i);
                    }
                }
        }


    return 0;
}
```

```
/*check_prime() is a function which returns if the input number
is prime, otherwise it returns 0 */
int check_prime(int n)
{
    int flag = 1,j;

    for(j=2; j<=n/2; j++)
        {
        if(n%j==0)
            {
            flag = 0;
            break;
            }
        }
        return flag;
}
```

c. Write a program to determine the factorial of a positive integer using recursive function

Code:

```
# include <stdio.h>
int main( )
{
    int a, fact ;
    printf ( "Enter any number " ) ;
    scanf ( "%d", &a ) ;
    fact = rec (a) ;
    printf ( "Factorial value = %d\n", fact ) ;
    return 0 ;
}

int rec(int  x)
{

    int f ;
    if (x == 1)
        {
        return 1;
        }

    else
        {
        f = x * rec (x - 1) ;
        }
```

```
    return f;
}
```

5. **Results after running the example programs:**
   Screenshots of the results:

6. **Lab report assignments (submit source code and screenshots of results):**
   a. Write a C program to find whether a number is Palindrome or Not using functions.
   b. Write a C program to get the nth Fibonacci term using recursion.
   c. Write a C program to find the sum of digits of a given number using recursion.

7. **Comment/Discussion on the obtained results and discrepancies (if any).**

Experiment No. 5

Arrays and String in C Language

1. **Objective:** The experiment is designed to provide basic knowledge of Arrays and String in C Language. Students will be able to develop logics which will help them to create programs, applications in C for arrays and strings.

2. **Theoretical Background:** Arrays are data types found in most high level languages. The array allows the storage of a number of data values in one variable. The underlying concepts to arrays are derived from matrix algebra. Besides being used to represent matrices, arrays are also suited for a variety of other purposes because, in C, this data type represents a defined number of contiguous (i.e. consecutive) bytes in memory.

   An array is the grouping of repetitive data (of the same type) that shares the same name. Arrays are useful when manipulating lists of data. The data shares the same name but each element in the array can be accessed individually. An array can be of any data type. Each constituent value of an array is called an element. When an array is defined in C, the programmer specifies the number of elements in the array. The elements of an array can be ordered in different pattern schemes. A one-dimensional array can be conceptually viewed as a single string or row of a predefined number of values. For example: if we declare an array of 6 elements as

   int a[6];

   then what happens is shown below:

a[0] int1	a[1] int2	a[2] int3	a[3] int4	a[4] int5	a[5] int6

   Rules for Forming Arrays in C:

   1. An array name must be chosen according to the same rules as are used for naming any other variable.

   2. The name of the array cannot be the same as that of any other variable declared within the function.

   3. The size of the array (the number of elements) is specified using the subscript notation.

   4. The dimension used to declare an array must always be a positive integer constant, or an expression that can be evaluated to a constant when the program is compiled.

5. All elements of an array should be of the same type. In other words, if an array is declared to be type int, it cannot contain elements that are not of type int.

The following assignment statements are all valid in C:

```
x[0] = 14 ;

x[5] = x[14] -7;

x[7] = x [6] +8+x[5];

x[3] += 1;
```

Strings in C: Strings are defined as an array of characters. The difference between a character array and a string is that the string is terminated with a special character '\0'. Strings are actually a one-dimensional array of characters terminated by a null character '\0'. Thus a null-terminated string contains the characters that comprise the string followed by a null. When the compiler encounters a sequence of characters enclosed in the double quotation marks, it appends a null character \0 at the end. Before you can work with strings, you need to declare them first. Since string is an array of characters. You declare strings in a similar way like you do with arrays.

```
char a[6];
```

then what happens is shown below:

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
char1	char 2	char3	char4	char5	char6

The following declaration and initialization will create a string consisting of the word "Hi". To hold the null character at the end of the array, the size of the character array containing the string is one more than the number of characters in the word "Hi."

```
char a[3] = {'H','i','\0'};
```

If you follow the rule of array initialization then you can write the above statement as follows –

```
char a[ ] = "Hi";
```

a[0]	a[1]	a [2]
H	i	\0

In the above example, you do not place the null character at the end of a string constant. The C compiler automatically places the '\0' at the end of the string when it initializes the array. You can initialize strings in a number of ways -

```
char c[ ] = "abcd";

char c[50] = "abcd";

char c[ ] = {'a', 'b', 'c', 'd', '\0'};

char c[5] = {'a', 'b', 'c', 'd', '\0'};
```

Rules for Forming Strings in C:

1. A string name must be chosen according to the same rules as are used for naming any other variable.

2. The name of the string cannot be the same as that of any other variable declared within the function.

3. The size of the string (the number of elements) is specified using the subscript notation which should also accommodate the null character.

4. The dimension used to declare a string must always be a positive integer constant, or an expression that can be evaluated to a constant when the program is compiled.

5. All elements of a string should be of the character type. In other words, it cannot contain elements that are not of type char.

3. Software requirements:

- Any IDE that supports a C compiler (preferably Code::Blocks; any other IDE or text editor with a C compiler installed will also be accepted.)

4. Example Problems:

a. Find the sum of all elements of an array

Code:

```
#include <stdio.h>
int main ()
{
    int midterm[10]={2, 2, 3, 3, 4};
    int i;
    int total = 0;
    for(i=0; i<5; i++)
    {
        total = midterm [i]+total;
    }
    printf("The Total Marks is %d", total);
    return 0;
```

```
}
```

b. Write a program to print the following numbers in reverse order
22 91 65 33 19 47 49 88 94 39

Code:

```
#include <stdio.h>
int main()
{
    int a[10]={22, 91, 65, 33, 19, 47, 49, 88, 94, 39};
    int i;
    //for printing current order
    for(i=0; i<10; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
    //for printing reverse order
    for(i=9; i>=0; i--)
    {
        printf("%d ", a[i]);
    }
}
```

c. Take a string as input from the user and print every single character of it into a separate line. If user gives “Programming”, output should look like the following

P
r
o
g
r
a
m
m
i
n
g

Code:

```
#include <stdio.h>
int main()
{
    char a[50];
    int index;
    printf("Enter The String Value\n");
    gets(a);
    for(index = 0; a[index] != '\0'; index++)
    {
        printf("%c", a[index]);
        printf("\n");
    }
    return 0;
}
```

d. Write a program in C to copy one string to another string

Code:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char source[50];
    char destination[50];
    printf("Enter any string within 50 characters: ");
    gets(source);
    strcpy(destination, source);
     printf("Here  is  the  text  that  you  entered:  %s",
destination );
    return 0;
}
```

5. Results after running the example programs:

Screenshots of the results:

6. Lab report assignments (submit source code and screenshots of results):

a. Write a program in C using an array which finds the largest and smallest elements in a group of numbers.

b. Write a program in C to augment one string with another string. For example: String ‘a’ contains “foot”, string ‘b’ contains “ball”, now string ‘c’ should contain “football”.
c. Write a program to print a string in reverse order.
d. A palindrome is a word, number or other sequence of characters which reads the same backward as forward, such as ‘madam’. Write a C program which can determine if a word is palindrome or not.

7. Comment/Discussion on the obtained results and discrepancies (if any).

Experiment No. 6

Pointers and Dynamic Memory Allocation

1. **Objective:** This experiment is intended to verify the concepts of pointers and dynamic memory allocation concepts in the C programming language. Overall, the example problems include pointer basics, pointer with arrays, pointer with strings, and dynamic memory allocation.

2. **Theoretical Background:** A pointer is a variable that holds the memory address of another object. For example, if a variable called p contains the address of another variable called q, then p is said to point to q. Therefore if q is at location 100 in memory, then p would have the value 100. To declare a pointer variable, use this general form:

```
type *var-name;
```

Here, the type is the base type of the pointer. The base type specifies the type of object that the pointer can point to. Notice that the variable name is preceded by an asterisk. This tells the computer that a pointer variable is being created. For example, the following statement creates a pointer·to an integer:

```
int *p;
```

C contains two special pointer operators: ***** and **&**. The **&** operator returns the **address of** the variable it precedes. The ***** operator returns the **value stored at the address** that it precedes. The ***** pointer operator has no relationship to the multiplication operator, which uses the same symbol.

Pointers are required in -

a. Dynamic memory allocation (accessing heap memory section)
b. Function call by reference
c. File accessing
d. Accessing peripherals like monitors, printers, etc.

Dynamic Memory Allocation can be defined as a procedure in which the size of a data structure (like Array) is changed during the runtime. C provides some functions to achieve these tasks. There are 4 library functions provided by C defined under <stdlib.h> header file to facilitate dynamic memory allocation in C programming. They are:

- `malloc()`
- `calloc()`
- `free()`
- `realloc()`

The “malloc” or “memory allocation” method in C is used to dynamically allocate a single large block of memory with the specified size. It returns a pointer of type void which can be cast into a pointer of any form.

It doesn't Initialize memory at execution time so that it initializes each block with the default garbage value initially.

Syntax:

```
ptr = (cast-type*) malloc(byte-size)
```

For Example:

```
ptr = (int*) malloc(100 * sizeof(int));
```

Since the size of int is 4 bytes, this statement will allocate 400 bytes of memory. And, the pointer ptr holds the address of the first byte in the allocated memory.

3. Software requirements:

- Any IDE that supports a C compiler (preferably Code::Blocks; any other IDE or text editor with a C compiler installed will also be accepted.)

4. Example Problems:

a. Write a program to find the smallest among three numbers using pointers.

Code:

```
#include<stdio.h>

int main(){

    int a,b,c,*pa,*pb,*pc;
    pa=&a;
    pb=&b;
    pc=&c;

    printf("Enter three integers: ");
    scanf("%d%d%d",pa,pb,pc);

    if (*pa<*pb && *pa<*pc)
        printf("the smallest number is %d",*pa);
    else if (*pb<*pa && *pb<*pc)
        printf("the smallest number is %d",*pb);
    else
        printf("the smallest number is %d",*pc);
    return 0;
}
```

b. Write a program to find the sum of all the elements of an array using pointers

Code:

```
#include<stdio.h>
int main(){
    int a[100],n,sum=0;
```

```
    printf("Enter the number of elements of the array: ");
    scanf("%d",&n);

    printf("Enter the array elements");
    for(int i=0;i<n;i++){
        scanf("%d",(a+i));
    }

    for(int i=0;i<n;i++){
        sum=sum+*(a+i);
    }

    printf("The sum of all the elements is = %d",sum);
    return 0;
}
```

c. Write a function to accept a string and count the number of vowels present in the string [use pointers].

Code:

```
#include<stdio.h>

int vowelCount (char *p){

    int counter = 0,i=0;

    while(*(p+i)){

         if(*(p+i) == 'A' || *(p+i) == 'E' || *(p+i) == 'I'
|| *(p+i) == 'O' || *(p+i) == 'U' || *(p+i) == 'a' || *(p+i)
== 'e' || *(p+i) == 'i' || *(p+i) == 'o' || *(p+i) == 'u'){

            counter++;

        }

        i++;
    }

    return counter;
}

int main(){

    char str[100];
    int n;
```

```
    printf("Enter a string: ");
    scanf("%s",str);

    n = vowelCount(str);

    printf("The number of vowels in the string is %d",n);

    return 0;
}
```

d. Write a program to remove all the empty spaces from a sentence. For example, if the input string is “The way to get started is to quit talking and begin doing”, the output should be “Thewaytogetstartedistoquittalkingandbegindoing”. [Store the input string in the dynamic memory].

Code:

```
#include<stdio.h>
#include<stdlib.h>

int main(){

    char *p,*q;
    int i=0,j=0;
    p = (char *)malloc(100*sizeof(char));

    printf("Enter a sentence: ");
    gets(p);

    q = (char *)malloc(100*sizeof(char));

    for(i=0;*(p+i);i++){

        if (*(p+i)!=' '){
            *(q+j) = *(p+i);
            j++;
        }

    }
    *(q+j) = '\0';

    printf("the output string is = %s",q);

    free(q);
    free(p);
```

```
    return 0;
}
```

5. **Results after running the example programs:**
   Screenshots of the results:

6. **Lab report assignments (submit source code and screenshots of results):**
   a. Write a program to concatenate two strings using pointers.
   b. Write a function to copy one array to another by using pointers.
   c. Write a function to find out the dot multiplication of two vectors.

7. **Comment/Discussion on the obtained results and discrepancies (if any).**

Experiment No. 7

Familiarization with Structure

1. **Objective:** This experiment is intended to verify the concepts related to *structure* in the C programming language. Examples are set to help students fathom the concept meticulously.
2. **Theoretical Background:**

Structure is a user-defined datatype in C language which allows us to combine data of different types together. Structure helps to construct a complex data type which is more meaningful. It is somewhat similar to an Array, but an array holds data of similar type only. But structure on the other hand, can store data of any type, which is practical and more useful.

For example: If I have to write a program to store Student information, which will have Student's name, age, branch, permanent address, father's name etc, which included string values, integer values etc, how can I use arrays for this problem, I will require something which can hold data of different types together.
In structure, data is stored in the form of records.

Defining a structure
struct keyword is used to define a structure. struct defines a new data type which is a collection of primary and derived data types.

Syntax:

```
struct [structure_tag]
{
    //member variable 1
    //member variable 2
    //member variable 3
    ...
}[structure_variables];
```

As you can see in the syntax above, we start with the struct keyword, then it's optional to provide your structure a name, we suggest you to give it a name, then inside the curly braces, we have to mention all the member variables, which are nothing but normal C language variables of different types like int, float, array etc.
After the closing curly brace, we can specify one or more structure variables, again this is optional.
Note: The closing curly brace in the structure type declaration must be followed by a semicolon(;).

Example of Structure:

```
struct Student
{
    char name[25];
    int age;
    char branch[10];
    // F for female and M for male
    char gender;
};
```

Here struct Student declares a structure to hold the details of a student which consists of 4 data fields, namely name, age, branch and gender. These fields are called structure elements or members.
Each member can have different data types, like in this case, name is an array of char type and age is of int type etc. Student is the name of the structure and is called the structure tag.

Declaring Structure Variables

It is possible to declare variables of a structure, either along with structure definition or after the structure is defined. Structure variable declaration is similar to the declaration of any normal variable of any other datatype. Structure variables can be declared in following two ways:

1) Declaring Structure variables separately-

```
struct Student
{
    char name[25];
    int age;
    char branch[10];
    //F for female and M for male
    char gender;
};

struct Student S1, S2;      //declaring variables
of struct Student
```

2) Declaring Structure variables with structure definition-

```
struct Student
{
    char name[25];
    int age;
    char branch[10];
    //F for female and M for male
    char gender;
}S1, S2;
```

Here S1 and S2 are variables of structure Student. However this approach is not much recommended.

3. Software requirements:

- Any IDE that supports a C compiler (preferably Code::Blocks; any other IDE or text editor with a C compiler installed will also be accepted.)

4. Example Problems-

Example 1: Define a structure “complex” to read two complex numbers and perform addition, subtraction of these two complex numbers and display the result.

Code:

```
#include<stdio.h>
struct complex{
 float real;
 float img;
};
int main(){
 struct complex c1,c2,add,sub;
  printf("Enter a and b of the first complex number (a+ib)
\n");
 scanf(" %f %f",&c1.real,&c1.img);
  printf("Enter a and b of the second complex number (a+ib)
\n");
 scanf(" %f %f",&c2.real,&c2.img);
 add.real = c1.real+c2.real;
 add.img = c1.img+c2.img;
 sub.real = c1.real-c2.real;
 sub.img = c1.img-c2.img;
 printf("c1+c2 = %f + %fi\n",add.real,add.img);
 if(sub.img<0) printf("c1-c2 = %f %fi",sub.real,sub.img);
 else printf("c1-c2 = %f + %fi",sub.real,sub.img);
 return 0;
}
```

Example 2: Write a menu driven program in 'C' which shows the working of the library. The menu option should be:
i) Add book details.
ii) Display book details.
iii) List all books of a given author.
iv) Show the count of books in the library.
v) Exit.
Create a structure called library to hold Book ID, title of the book, author name, price of the book.
Code:

```
#include<stdio.h>
#include<string.h>
struct library{
 int id;
 char title[40];
 char author[20];
 float price;
} b[100] ;
int num=0;
void Add(){
 printf("How many books' info do you want to enter? ");
 scanf(" %d",&num);
 for(int i=0;i<num;i++){
 printf("Enter the following information about the book:\n");
 printf("ID, title, author's name, price(in Tk)\n");
 scanf(" %d %s %s
%f",&b[i].id,&b[i].title,&b[i].author,&b[i].price);
 }
}
void Disp(){
 printf("\tID\tName\tAuthor\tPrice(Tk)\n");
 for(int i=0;i<num;i++){

printf("\t%d\t%s\t%s\t%f\n",b[i].id,b[i].title,b[i].author,b[i].p
rice);
 }
}
void Count(){
 printf("\nNo of books avalable in the library = %d\n",num);
}
void List(){
 char str[20];
 printf("Enter the author's name: ");
 scanf("%s",str);
 for(int i=0;i<num;i++){
 if(strcmp(str,b[i].author)==0)

printf("\n\t%d\t%s\t%s\t%f\n",b[i].id,b[i].title,b[i].author,b[i]
.price);
 }
}
int main(){
 int option=0;
 do {
 printf("\nWelcome to the library\nPlease Select an Option: \n");
```

```
printf("------------------------------------------------------------
--
\n");
 printf("1.Add book details\n2.Display book details\n3.List all
books of a given author\n4.Show total no. of books in the
library.\n5.Exit\n");

printf("------------------------------------------------------------
--
\n");
 scanf("%d",&option);
 switch(option){
 case 1: Add();
 break;
 case 2: Disp();
 break;
 case 3: List();
 break;
 case 4: Count();
 break;
 }
 }while(option != 5);
 return 0;
}
```

5. Lab Report:

Make the following modifications to the system in Example-2 -

- Add a password-protected authorization system so that option-1 can be used by the librarian only
- Enable the use of multi-word strings in book titles and author names
- Make the system dynamic: make sure that the newly input book info does not overwrite the previous ones